Polymorphism
(Method Overriding)

What is Polymorphism?

- Same method name
- Different behavior

Real-World Example

- Animal → sound()
  - Dog → bark
  - Cat → meow

In Shopping malls behave like Customer

In Bus behave like Passenger

In School behave like Student

At Home behave like Son Sitesbay.com

What is Polymorphism?

Polymorphism means the **same method name can behave differently** depending on:

- the object calling it, or
- the data passed to it.

Types of Polymorphism in Python

Type	Key Idea	Example
Method Overloading	Same method, different arguments	add(a,b,c)
Method Overriding	Child changes parent method	sound()
Operator Overloading	Operators behave differently	+
Duck Typing	Method existence matters	fly()

Method Overloading
(Compile-time Polymorphism – concept based)

- Same method name, **different number or type of arguments**.
- Python does **NOT support true method overloading** like Java. But we achieve it using **default arguments or *args**.

```python
class Calculator:
    def add(self, a, b=0, c=0):
        return a + b + c

calc = Calculator()

print(calc.add(5))
print(calc.add(5, 10))
print(calc.add(5, 10, 15))
```

```
5
15
30
```

- Same method add()
- Works differently based on arguments

Method Overriding
(Runtime Polymorphism)

- Child class **redefines** a method from the parent class.

```python
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

class Cat(Animal):
    def sound(self):
        print("Cat meows")

dog = Dog()
cat = Cat()

dog.sound()
cat.sound()
```

```
Dog barks
Cat meows
```

- Same method sound()
- Different behavior in child classes Dog and Cat, so parent class behavior is overridden

Operator Overloading

- Same operator behaves **differently for different data types**.

```python
print(10 + 20)
print("Hello " + "World")
```

→

```
30
Hello World
```

- Same + operator
- Adds numbers
- Join strings

Custom Operator Overloading (Dunder Methods)

```python
class Book:
    def __init__(self, pages):
        self.pages = pages

    def __add__(self, other):
        return self.pages + other.pages

b1 = Book(100)
b2 = Book(200)

print(b1 + b2)
```

→

```
300
```

Duck Typing
(Python-specific Polymorphism)

- If it looks like a duck and quacks like a duck, it is a duck.
- Python **does not care about class type**, only whether the method exists.

```python
class Bird:
    def fly(self):
        print("Bird can fly")

class Airplane:
    def fly(self):
        print("Airplane can fly")

def make_it_fly(obj):
    obj.fly()

b = Bird()
a = Airplane()

make_it_fly(b)
make_it_fly(a)
```

```
Bird can fly
Airplane can fly
```

- No inheritance
- Same method name
- Python just checks **method existence**

Duck Typing
(Python-specific Polymorphism)

```python
class Bird:
    def fly(self):
        print("The bird is flying.")

class Plane:
    def fly(self):
        print("The plane is flying.")

class Whale:
    def swim(self):
        print("The whale is swimming.")

def make_it_fly(entity):
    # Python doesn't check type here, it just tries to call the fly() method
    try:
        entity.fly()
    except AttributeError as e:
        print(f"Error: {entity.__class__.__name__} cannot fly. Details: {e}")

# Usage
sparrow = Bird()
airplane = Plane()
whale = Whale()

make_it_fly(sparrow)  # Output: The bird is flying.
make_it_fly(airplane) # Output: The plane is flying.
make_it_fly(whale)    # Output: Error: Whale cannot fly. Details: 'Whale' object
    has no attribute 'fly'
```